SPEC-DRIVEN DEVELOPMENT · 施工图 No.TANG-001

我不写代码，我写规约。

一个人 + 两个 agent,6 天做出一个 HTAP 数据库

代码由 agent 按规约实现,我负责独立核验;
核验不过,不算做完。

仓库
github.com/1638540149/tang_db

2026-10 · tang_db 复盘 · VIBECODING.md

先说结果:下面每个数字我都能当场重跑

6 天
P1 到 P4 落地
P5 施工中

9 次
提交次数
一期一个提交

7 份
分期规约
P1~P5 + 2 个补丁期

395 条
功能需求 FR
另有 SC 113 条

260 个
FE 单测全绿
P4 验收时

0 条
强杀后数据丢失
1000 条 KV 崩溃测试

代码量:BE Go 7,689 行 · FE Java 16,826 行 · 测试 5,611 行 · 测试脚本 3,286 行

取数方式:git log · wc -l · docs/验收报告/P4-验收报告.md

vibe coding 不等于 vibe in production

只给目标:① 给我实现一个数据库

- × 800 行看起来能跑的代码,不知道错在哪
- × 没人能回答「做完了没有」
- × 强杀进程后数据丢了,也没人发现
- × 改动无法审:diff 里全是「顺手优化」

给规约:② 这份文档,照着做

- ✓ 每条要求有编号,做完没做完一眼可见
- ✓ 每条验收能翻译成一行断言,失败就红
- ✓ 边界写死,AI 没有「顺手发挥」的空间
- ✓ 换一个 agent 也能做出同样的东西

AI 不是瓶颈了。

评审和集成才是 —— 写 1000 行代码只要一个上午,判断它对不对要一个人日。

流程不是拍出来的:白皮书 → skill → 项目

① 白皮书 38 页

《Spec-Driven Production Grade Development in the Age of Vibe Coding》Lee Boonstra · Google · 2026-05

② 提炼成 skill

把抽象主张落成工序:选题 → 设计方案 → 逐项拍板 → 分期规约 → 交实现 → 独立核验

③ 落到项目

7 份规约 · 4 份验收报告 · 每期一份交接单

FIG.1 **落下去的才叫采纳:**SDD / 好规格四要素 / 格式选择 / BDD 场景 / 指令住在哪 / 不同用途不同 prompt / 版本号写死 / 护栏与人工在环 / AI 生成测试覆盖

FIG.2 **没采纳的也写清:**Evaluation 打分 / 容忍带 —— 数据库输出是确定的,用不变量与双路径对账替代;MCP、Tier3 自动评审 —— 单人单机,由「换人核验 + 重跑命令」替代。

把方法论逐节标「采纳 / 不采纳」,比假装全用上更能说明白它到底管什么。

三个人干一件事:写的、验的、拍板的,不许合并

我 • 项目所有者

- 定方向、逐项拍板
- 只有我能把规约 Draft → Approved
- 看验收报告与读代码地图

不做:不写实现、不写测试

Hermes • 规约 + 核验

- 起草分期规约、评审实现
- 独立重跑全部验证
- 写验收报告与读代码地图

不做:不自己批准规约、不代改代码

opencode • 编码

- 按规约写代码与测试
- 真跑构建与测试
- 给出真实输出与未验证项

不做:不碰规约状态、不 commit

写代码的那个 agent,不许给自己打分。

核验必须换会话、换上下文、只看仓库现状。

什么该住在哪里:五层指令,放错层就失效

层	放什么	谁读 / 什么时候读	放错的后果
个人记忆层	机器、路径、端口、我的偏好	agent 每个会话自动带	写进仓库 = 泄露 + 噪音
方法层(skill)	跨任务不变的工序	按需加载,一处生效	写进 prompt = 每期重抄
项目层(AGENTS.md)	硬约束、构建命令、提交纪律	每次进仓库先读	不写 = agent 只能靠猜
任务层(specs/Pn.md)	这一期要什么	编码前通读一份	塞进聊天 = 上下文碎片化
本轮层(.pN-prompt)	这一次怎么干	只对这一次有效	当归档用 = 下期照抄过期规则

只想让它读一份:把 100 页设计文档直接塞进聊天,会烧掉上下文、拉高延迟、把上下文撕碎。

```
1 AGENTS.md → 硬约束(每次读)
2 specs/README.md → 找到当前期
3 specs/Pn-*.md → 唯一施工依据(通读)
```

只给一份规约

给两份,它就开始自己融合、自己发明。

一期七步:只有第 5 步交给 AI

① **立规约** Draft

② **逐项拍板** 回字母

③ **批准** 只有我能批

④ **写交接单** .pN-prompt

⑤ **编码** agent 实现

⑥ **独立核验** 重跑命令

⑦ **归档** 报告 + 地图

↺ 核验有红 → 回第 4 步重写交接单

↺ 实机发现本期缺口 → 回第 1 步升规约版本

FIG.1 **为什么一期一停:**AI 的失真是累积的 —— 第一期边界穿了一毫米,到第四期就是整层塌掉。每期都能独立跑、独立演示、独立验收,上一期没跑通就不开工。

FIG.2 **期次是拆出来的:**P2.1 / P3.5 是「一期做大了就拆」;P3.5 原排 P8,因「只用自带 CLI 太麻烦」提前,又必须先有常驻服务形态,于是再拆三步。

一句话

前面四步决定它能不能做对,后面两步决定它做的算不算数。

一份规约的十二节骨架,和五套编号

1 目标:比上期多了什么

2 范围:做 / 不做及归属期次

3 用户场景:BDD 写法 + ★

4 功能需求:FR 编号

5 数据与格式:布局 / 编码 / 错误码

6 外部评审必答项:正面回答

7 关键技术决策:K 编号

8 验收标准:SC 编号

9 边界与异常清单:逐条编号

10 交付物清单

11 代码由谁写:角色分工

12 待拍板 + 13 变更记录

FR	要求:一条一行,可观测	395 条
SC	验收:带判定方式	113 条
K	决策:默认值 + 理由	60 条
边界	异常:输入/客户端/崩溃	P4 共 40 条
INV	不变量:跨期,不许退化	18 条

编号不是好看:规约写 FR-555~558 → 脚本就有 SC-344/345 → 测试就有对应用例。缺一环,报告里就露出来。

编号跨期续排:P4 从 FR-456 起(因为 P3.5 用到 FR-455),补丁期不许从 1 重开。

写规约的七条硬规则

① 每条要求一个编号	没有编号的条款,验收时一定漏
② 判定必须可观测	写「返回 1062、信息含索引名、表里仍 1 行」,不写「正确处理」
③ 每个场景必须有反例	只写「那么报错」,AI 会写出「报错但数据也写进去」
④ 「不做」要穷举 + 写归属	谓词 NOT/LIKE/BETWEEN 曾因无归属,被实机当成一堆 bug 报回来 → 现明确归 P5
⑤ 数据格式写到字节	key 布局 / value / 编码 / 错误码 —— 它唯一没有自由发挥空间的地方
⑥ 版本号端口路径写死	模型知识有截止日,库版本不能靠它记(白皮书专门提醒)
⑦ 状态只有人能改	编码中发现规约缺口,用代码覆盖规约是最严重的违规

两条自检:规约里不许留 `FR-xxx` 占位;数量声明先 `grep` 一遍再写进变更记录。

四跳链路:场景 → FR → SC → 断言,缺一跳等于没验

① 场景 §3
给 / 当 / 那么 + 反例

② FR §4
FR-484 整批不写、报 1062

③ SC §8
SC-321 ★ 单会话挡重复

④ 断言
p4-test.sh 6 条 expect

```
OUT="$(mysql_code "INSERT INTO t VALUES (2,'A001')")"
expect "${OUT}" "CODE=1062"    "重复插入返回 1062"
expect "${OUT}" "uk_code"      "错误信息含索引名"
expect "${OUT}" "code"         "错误信息含列名"
expect "${OUT}" "A001"         "错误信息含冲突值"
expect "${OUT}" "已有主键 1"   "错误信息含已有主键"
# 反例也断言:冲突后表里仍只有 1 行
```

★ 只打两类
并发、崩溃 —— 平时跑不出来,一旦不对就是数据已错

判据只有一句
这个场景能不能翻译成一行 expect?不能就重写

场景 11 更狠:5000 次随机 INSERT / UPDATE(含改索引列)/ DELETE,索引路径与全表扫的主键集合差异必须为 0(SC-329 / INV-10)。
它一条同时验三件事:写入维护索引、删除写墓碑、读取跳过墓碑 —— 任一处漏了,差异就不是 0。

```
$ bash scripts/tests/p4-test.sh   →   PASS=118  FAIL=0  NOTE=4
```

施工交接单:六段,把解释空间压到零

① 必读顺序

只给一份规约当唯一依据

② 实现顺序

按模块分步,每步跑构建

③ 硬约束

每次重述,重复比返工便宜

④ 验证要求

写到命令级,含输出格式

⑤ 不要做的事

含「宁可停下来报告」

⑥ 环境细节

端口 / 日志 / 不许前台阻塞

- 不要伪造或推测任何测试结果。没跑就写「未验证」。
- 不要超出本期规约范围(不顺手做别的期次)。
- 遇到无法解决的阻塞,停下来在最终输出里说明,
  不要硬编一个看起来能跑的实现。

最后一句是关键

AI 面对阻塞的本能是「编一个看起来能跑的版本」,而这个假实现会污染后面所有期次。给它一条体面的退路:停下来报告,比糊过去得分高。

补丁期更严:写清上游状态 + 「只做这两条,不要顺手做别的」,每条到文件级与行级依据。

该我决定的，一条一条问：一次一问，回字母

编号	要拍的事	我的建议	结论
I16	回填期间的新写入怎么不丢索引?	写入路径对 BUILDING 索引也维护	已拍板 A
I17	唯一索引的 NULL 语义	不索引 NULL 行,允许多行 NULL	已拍板(按建议)
I22	SHOW ENGINES 报 InnoDB 算不算欺骗?	报,README 写清「仅为客户端兼容」	已拍板(按建议)

P4 的八项(I15~I22)就是这样一条条拍死的。

不要在聊天里甩一张十项的表让人填 —— 那不是征求意见,是甩锅。

拍完 → 原样回写规约 §12(带日期)→ 状态转 Approved → 才允许开工

规约状态只有我能改。任何 agent(包括起草的那个)不得自行批准。

核验 SOP:不信任的部分,全部自己重跑

1. 清残留进程(be.exe / fed),否则端口被占 → 整轮假红
2. 用独立高位端口:BE 9900 / MySQL 9906 / 心跳 9991
3. go build ./... && go vet ./... && go test ./...
4. mvn.cmd test,看用例总数与失败数
5. 一键验收:拿 PASS/FAIL 真数,内嵌往期回归也要绿
6. 崩溃注入:强杀 → 重启 → 逐条校验
7. 真客户端抽检(fed + Navicat / MysqlErrorCheck)
8. 边界零改动:git status --short be/ proto/ 必须为空
9. 写报告:输出原样摘录,没验的写「未验证」

宪法第四条:凡声称测试已通过,必须能给出实际执行输出。

报告三条纪律

① 数字来自我本人的终端
② 输出原样摘录,不美化
③ 没跑就写「未验证」

P4 报告的真实结论段

「结论:通过。…… 未验证项已明示(GUI 全流程、Navicat 本机未装、大表回填性能)。」

数字必须对得上:断言、边界、丢失条数

97 → 118

P4 断言增量 +21

正好对应 G 组两条 SC

0 改动

BE 与 proto 零改动

git status --short be/ proto/ 为空

0 条

强杀后数据丢失

1000 条随机 KV 崩溃测试

为什么盯这三个数:

断言增量对得上 SC ——否则就是「功能说做了,其实没验」;

边界零改动 ——规约说本期活全在 FE,那 BE 动了就是越界,不能靠「看起来没事」带过;

丢失 0 条 ——「先日志后数据」被偷偷改成异步,只有强杀能发现。

```
$ bash scripts/tests/p4-test.sh        →  PASS=118  FAIL=0  NOTE=4
  内嵌回归: p2-test.sh 52/0   p3-test.sh 62/0   p35-test.sh 46/0
$ bash scripts/tests/crash-test.sh     →  1000 条随机 KV,强杀重启,丢失 0 条
```

把真实客户端连上去, 才知道缺什么

Navicat 连上
连接期报 1235

→

我拍板
属本期就本期补

→

规约升版
v0.1f → v0.1i

→

补丁交接单
只做这几条

→

回归全绿
118 / 0

`LIMIT 0,1000`	所有图形客户端翻页都这么写 → 原排 P5,提到本期(K100)
两列排序	拉表清单用 ORDER BY schema,type → 只做单列就报错(K99)
3 条语句一个包	Navicat 连接期自发检 → 单包多语句(FR-555~558)
`SHOW PROCEDURE STATUS`	打开每个库都发一条 → 返回 11 列空集(FR-559)

这一层 AI 自己想不到
它不知道 Navicat 会发什么 —— 只有我真连上去、真去点,缺口才会浮出来。

八种翻车方式, 以及对应的挡板

失败模式	症状	挡板
规约写成许愿单	条款翻译不成一行 expect	一条 FR 必须带判定方式
规约与代码不同步	agent 幻觉出不存在的函数	变更与代码同轮交付;改名后全库 grep
一份规约管太多期	上下文碎片化,条款互相冲突	一期一停,独立可验收
自己批准自己	报告好看但是假	状态权限只在人;核验换 agent
用形容词当口径	「性能良好」「正确处理」	换成数字 / 错误码 / 差异计数
评审意见记成待办	文档越来越长,问题没解决	当场改正文;不吸收就写理由
agent 顺手重构	diff 大到无法审	交接单只做这几条;补丁期更窄
期次顺序≠施工顺序	某步验收依赖后一步产物	交接单写清提前量,不改规约正文顺序

共同点:它们都不报错,只是让「做完了」变得不可验证。

工具链:skill 管方法,命令管动作,插件管编排

Hermes • skill

— spec-driven-development-workflow

— `tang-db-phase-delivery v1.4.0`

— technical-design-doc

— `speckit-* 10 个`

— opencode-run-delegation-pitfall

Spec Kit • opencode 命令

— `speckit.specify / plan / tasks`

— `speckit.implement / converge`

— `speckit.clarify / analyze`

— `speckit.checklist / constitution`

— 集成 1.2.2,2026-10-21 装入

opencode • 插件

— `oh-my-openagent(subagent 编排)`

— `@opencode-ai/plugin 1.19.0`

— → 所以要明写「不要派子代理」

— opencode run -m deepseek-flash

— 日志落 tmp/*.log 便于探活

为什么用 skill 而不是每次写长 prompt:

① 复用 ——方法论跨项目不变,不该每开一个仓库重写;
② 可演进 ——踩过的坑当场写回 skill(脚本布局硬约定 / 别按名字杀 java.exe / PASS 降 NOTE);
③ 一处生效 ——改了 skill,后面每一期都按新规矩走;写在 prompt 里的只对那一期有效。

最小配方 14 条,仓库里都能翻到原件

不是「让 AI 随便写」,是「让 AI 写、让命令说话」。

01 设计文档先行,再写宪法(5~6 条硬约束)

02 文档送外部模型评审,要求章节级 + 反例

03 当期答不了的登记成必答项 + 归属期次

04 路线切成一期一停,每期可独立演示

05 每期一份规约:做 / 不做 / 场景 / FR / SC

06 待拍板一次一问,拍完回写

07 只有人能批准 Draft → Approved

08 交接单六段:必读 / 顺序 / 约束 / 验证 / 禁区 / 环境

09 编码与核验必须换 agent

10 核验含构建、单测、一键脚本、崩溃注入、真客户端

11 写验收报告(未验项明示)+ 读代码地图

12 实机缺口 → 补丁期 → 规约升版本

13 一期一个提交,写清「做了什么 + 怎么验的」

14 踩过的坑写回 skill,下一期自动继承

仓库里对应的原件

`VIBECODING.md`
整套方法复盘

`specs/`
7 份规约 + 交接单 + 白皮书

`docs/`
设计方案 / 验收报告 / 读代码地图

`scripts/tests/`
断言即回归基线